



技术参考指南

IT实用工具包的背景概念

移动应用开发

JP2026 7月线上友谊赛

模块 A(KR) - 电话

本文档为补充参考资料，解释了每种工具背后的概念和计算方法，以便非密码学或网络学专家的应用程序开发人员能够正确地实现测试项目。本文档不包含解决方案源代码。

内容

目录.....	2
1. 引言.....	4
2. 基础知识：字节、十六进制和位运算.....	4
2.1 字节和十六进制.....	4
2.2 位运算.....	4
3. 安全与密码学.....	5
3.1 哈希（哈希生成器、文件哈希）	5
盐.....	5
文件哈希.....	6
3.2 HMAC（基于哈希的消息认证码）	6
3.3 AES 对称加密.....	6
密码模式：数据块的链接方式.....	6
初始化向量（IV）	6
填充（PKCS7）	7
综合起来.....	7
4. 编码和转换.....	7
4.1 字符编码：ASCII 和 UTF-8.....	7
4.2 Base64	7
4.3 URL 编码（百分比编码，RFC 3986）	8
4.4 数字进制转换器.....	8
4.5 ASCII / HEX 转换器	8
5. 人脉网络.....	9
5.1 IPv4 地址	9
5.2 子网掩码和 CIDR 表示法	9
5.3 CIDR 计算步骤.....	9
示例 1 — 192.168.1.10 /24	9
示例 2 — 172.16.5.130 /27（八位字节内的子网划分）	10
子网掩码快速参考.....	10
5.4 IP地址计算器.....	10
5.5 端口（端口检查器）	10
6. 通用工具.....	11
6.1 UUID 生成器	11
6.2 密码生成器.....	11
6.3 正则表达式测试器	11
6.4 随机数生成器.....	12

附录 A. 快速参考表..... 13

 A.1 二的幂..... 13

 A.2 十六进制↔二进制半字节..... 13

 A.3 部分可打印 ASCII 码 13

 A.4 样本测试向量中使用的值..... 13

1. 引言

IT实用工具包包含15个IT开发人员和安全工程师日常使用的实用工具。每个工具界面看起来都很简单，但其中一些工具依赖于密码学、字符编码和IP网络领域的标准算法。本指南将介绍这些算法的功能、存在意义以及结果的计算方法，并提供与提供的test-vectors.json文件中的值相匹配的示例。

如何阅读本指南：

- 每个章节首先用浅显易懂的语言解释概念。
- “计算方法”小节给出了具体的计算方法，并一步一步地进行了说明。
- 示例代码使用与应用程序原型中相同的输入。
- 文末的参考表格总结了最常用的值（2 的幂、子网掩码、常用端口、ASCII 码）。

关于表征的说明。在整个工具包中，有三个概念反复出现：

- 一切皆字节。文本、文件、密钥和 IP 地址最终都是由 8 位字节（值 0 到 10²⁵⁵）组成的序列。
- 文本经过编码后会变成字节。此工具包始终使用 UTF-8 编码。
- 字节以十六进制（以 16 为基数）或 Base64 的形式向人类显示，因为原始字节是不可打印的。

2. 基础知识：字节、十六进制和位运算

几乎所有工具都会用到这些基本概念，它们对于本指南后面的网络计算至关重要。

2.1 字节和十六进制

一个字节存储 0 到 255 之间的值。用二进制表示为 8 位，例如 1100 0000。由于长的二进制字符串难以阅读，一个字节通常写成两个十六进制数字，其中每个数字代表 4 位（“半字节”）。

二进制（4 位）	十六进制	小数
0000	0	0
0101	5	5
1010	一个	10
1100	C	12
1111	F	15

例如：字节1100 0000 =十六进制C0 =十进制192.

2.2 位运算

位运算符作用于数值的各个比特位。其中三个是 CIDR 计算的核心。

操作员	规则	示例（每比特）
和（&）	当且仅当两个位均为 1 时，结果位才为 1。	1100 和 1010 等于 1000
或（ ）	如果任一一位为 1，则结果位为 1。	1100 1010 = 1110
不（~）	将每一位取反（0↔1）	~1100 = 0011
向左移动（<<）	向左移动位，填充为 0。	0001 << 3 = 1000

“与”运算用于保留某些位并将其余位置零（“掩码”）。“或”运算用于强制位为1。这些步骤正是用于查找网络地址和广播地址的方法。

3. 安全与密码学

3.1 哈希（哈希生成器、文件哈希）

哈希函数接受任何输入——一个单词、一段文字或整个文件——并生成一个固定大小的“指纹”，称为摘要。相同的输入总是产生相同的摘要，但你无法从摘要反推输入。

主要特性：

- 确定性：相同的输入总是产生相同的输出。
- 固定大小：输出长度仅取决于算法，与输入大小无关。
- 单向性：将摘要逆向还原为输入在计算上是不可行的。
- 雪崩效应：改变一个字符，大约会改变一半的输出位。
- 抗碰撞性：应该很难找到两个具有相同摘要的不同输入。

典型用途包括文件完整性检查（此下载是否已被篡改？）、存储密码验证器和去重。

算法及其输出规模：

算法	文摘尺寸	十六进制字符	笔记
MD5	128 位（16 字节）	32	速度快；但碰撞检测存在问题——仅用于校验和计算。
SHA-1	160 位（20 字节）	40	出于安全考虑，已弃用
SHA-224	224 位（28 字节）	56	SHA-2 系列
SHA-256	256 位（32 字节）	64	SHA-2 系列；如今非常常见
SHA-384	384 位（48 字节）	96	SHA-2 系列
SHA-512	512 位（64 字节）	128	SHA-2 系列

摘要是一串字节；工具包将其显示为小写十六进制数。例如，MD5 摘要的 16 个字节会变成 32 个十六进制字符。

示例——MD5世界技能大赛：

输入	世界技能大赛	(UTF-8 字节: 57 6F 72 6C 64 53 6B 69 6C 6C 73)
MD5	:	e514b0a3298d6efe1322b85ddb91a791
SHA-256:		7a474640b9e0fc9aa42fd27d92f5ffe331b25b0ab5e0537d2bd6d7ee387ffd75

盐

盐值是一个短的随机值，在哈希运算之前与输入数据组合在一起。由于每次哈希运算的盐值都不同，因此两个相同的输入会产生不同的摘要，从而可以抵御预先计算的“彩虹表”攻击。在本工具包中，盐值是一个随机的 16 字节值，以 32 个十六进制字符的形式显示，并在哈希运算之前以文本形式与输入数据组合在一起：

hashInput = UTF8(inputText + displayedSaltText) digest =
SHA-256(hashInput)

文件哈希

文件哈希运算的工作原理与文本哈希运算完全相同，区别在于输入的字节来自文件内容，而不是输入的字符。将新计算的哈希值与已发布的哈希值进行比较，可以证明文件下载时未被损坏或篡改。应用程序中显示的文件大小为字节数 ÷ 1024，并四舍五入到小数点后两位（例如，49 字节 = 0.05 KB）。

3.2 HMAC（基于哈希的消息认证码）

普通的哈希值可以证明数据没有被篡改，但任何人都可以计算它，因此无法证明数据的创建者。HMAC 通过在哈希过程中加入一个密钥来解决这个问题。只有知道密钥的人才能生成或验证正确的哈希值。因此，HMAC 既能保证数据的完整性，又能保证数据的真实性。

从概念上讲，消息和密钥会组合在一起并进行两次哈希处理：

HMAC(密钥, 消息) =
H((key XOR outerPad) + H((key XOR innerPad) + message))

“H” 代表任意哈希函数，因此该工具提供HMAC-MD5、HMAC-SHA1、HMAC-SHA256和HMAC-SHA512等哈希函数。实际应用包括对API请求进行签名、验证Webhook以及JSON Web Tokens (JWT) 的签名部分。输入和密钥均采用UTF-8编码，结果以小写十六进制形式显示。

3.3 AES对称加密

AES（高级加密标准）是应用最广泛的对称加密算法。“对称”意味着加密和解密使用同一个密钥。与哈希不同，加密是可逆的：使用正确的密钥可以完全恢复原始文本。

AES 以固定的 16 字节（128 位）块处理数据。密钥长度可以是 128 位、192 位或 256 位（16 字节、24 字节或 32 字节）；密钥越长，内部轮数越多，密钥空间也越大。

算法	关键长度	块大小
AES-128	16 字节	16 字节
AES-192	24 字节	16 字节
AES-256	32 字节	16 字节

密码模式：数据块的链接方式

分组密码只能加密一个 16 字节的数据块。“操作模式”定义了如何将较长的消息分割成多个数据块并进行组合。该工具包提供了两种经典模式。

- 电子密码本 (ECB)：每个数据块独立加密。简单但相同的明文块会生成相同的密文块，这会导致模式泄露。它不使用初始化向量 (IV)，因此不建议用于实际数据——此处仅用于学习和比较。
- CBC（密码块链接）算法：每个数据块在加密之前都会与前一个密文块进行异或运算。第一个数据块没有对应的前一个数据块，因此它与初始化向量（IV）进行异或运算。这使得相同的明文生成不同的密文，从而隐藏了其中的规律。

CBC 加密：
C0 = Encrypt(P0 XOR IV) C1 =
Encrypt(P1 XOR C0) C2 =
Encrypt(P2 XOR C1) 。 。 。 等等

初始化向量（IV）

初始化向量 (IV) 是一个 16 字节的值（与一个数据块的大小相同）。它不需要保密，通常与密文一起存储或发送，但对于 CBC 加密算法，IV 应该是不可预测的，并且每次加密都不同。在本工具包中，IV 以 32 个十六进制字符（16 字节 × 2 个十六进制数字）的形式输入。IV 字段仅在需要时显示。

CBC 模式；欧洲央行不使用 CBC 模式。要解密 CBC 消息，必须提供与加密时相同的初始化向量 (IV)。

填充 (PKCS7)

由于 AES 加密算法以 16 字节为单位进行加密，因此明文必须填充至 16 字节的倍数。PKCS7 填充算法会添加 N 个字节，每个字节的值为 N，其中 N 是所需的填充字节数 (1-16)。如果数据已经是 16 的倍数，则会添加一个值为 16 的完整额外字节块 (0x10)，以便解密时始终能够明确地移除填充字节。

把它们组合起来

该工具使用的加密流程如下：

1. 将纯文本转换为 UTF-8 字节。
2. 使用 PKCS7 将字节填充为 16 的倍数。
3. 使用选定的密钥长度和模式进行加密 (CBC 使用 IV；ECB 不使用)。
4. 将生成的密文字节编码为标准 Base64 (无换行符) 以便显示。

解密过程与此相反：对输入进行 Base64 解码，解密块，移除 PKCS7 填充，并将字节读回为 UTF-8 文本。

示例 — AES-128 / CBC：

```
纯文本: Hello WorldSkills 密钥
(ASCII) : 1234567890ABCDEF          (16 字节 → AES-128)
IV (十六进制) : 000102030405060708090A0B0C0D0E0F 密文:
n2tQ3xoA0rRAUoInZAeHLBjD4S6g0VhQwrHfkXcl/Hw=          (Base64)
```

如果重复运行相同的输入，你会得到完全相同的 Base64 字符串。改变初始化向量 (IV)，即使密钥和文本相同，密文也会完全改变。

4. 编码和转换

编码并非加密。它改变数据的表示形式，使其能够安全地通过纯文本通道传输；任何人都可以逆向还原。此过程无需密钥。

4.1 字符编码：ASCII 和 UTF-8

计算机将文本存储为称为代码点的数字。ASCII 定义了前 128 个代码点 (0-127)：英文字母、数字和常用标点符号。UTF-8 是 ASCII 的超集，它可以使用每个字符 1 到 4 个字节来表示任何语言的每个字符；前 128 个代码点与 ASCII 相同 (每个字符 1 个字节)。此工具包全部使用 UTF-8，因此 ASCII/HEX 工具将输入限制在 ASCII 范围内，而哈希和加密功能则接受任何 UTF-8 文本。

4.2 Base64

Base64 使用 64 个安全字符 (A-Z、a-z、0-9、“+”和“/”) 将任意字节转换为可打印文本。它用于将二进制数据 (例如图像、密钥、密文) 嵌入到 JSON、电子邮件或 URL 等文本格式中。

计算方法：每次取三个输入字节。三个字节共 24 位；这 24 位被重新组合成四个 6 位数，每个 6 位数 (0-63) 从字母表中选择一个字符。当输入长度不是 3 的倍数时，会在末尾添加一个或两个 “=” 符号作为填充。

剩余输入字节	输出	填充
3 字节	4 个字符	没有任何
2 字节	3 个字符 + "="	一个 "="
1 字节	2 个字符 + "=="	两个 "="

示例：2026年世界技能大赛→V29ybGRTa2lsbHMgMjAyNg==

该工具包使用标准字母表（而非 URL 安全的“-”变体），并且从不插入换行符。解码过程会反转映射关系；无效的 Base64 输入或无法解码为有效 UTF-8 文本的输入将被拒绝并返回错误。

4.3 URL 编码（百分比编码，RFC 3986）

URL 只能包含有限的字符集。百分比编码会将每个不安全的字节替换为“%”及其对应的两位十六进制值。此处遵循的规则来自 RFC 3986：

- 未保留字符保持不变：A-Z、a-z、0-9、连字符“-”、句点“.”、下划线“_”和波浪号“~”。
- 每隔一个字节使用其 UTF-8 值进行百分比编码，使用大写十六进制数字（例如“/”→%2F）。
- 空格会变成 %20。（“+”号仅在 HTML 表单提交中表示空格，规则不同——这里不使用。）

示例：

输入：https://worldskills.org/?q=mobile app&lang=ko
输出：https%3A%2F%2Fworldskills.org%2F%3Fq%3Dmobile%20app%26lang%3Dko

4.4 数字进制转换器

数字可以用不同的进制表示。在任何进制 b 中，每个数字位置的值都是 b 的幂。该工具包可在四种进制之间进行转换：

根据	姓名	使用的数字	每数字位数
2	二进制	0-1	1
8	八进制	0-7	3
10	小数	0-9	—
16	十六进制	0-9 和 A-F	4

计算方法如下：要转换成十进制，将每个数字乘以其对应的位值，然后将所有乘积相加。要从十进制转换成其他进制，需要反复除以目标进制，并从下往上读取余数。十六进制与二进制的对应关系很完美，因为一个十六进制数字正好等于四位二进制数，而一个八进制数字等于三位二进制数。

示例——二进制数1111111：

1111111（二进制）= 1 · 64 + 1 · 32 + 1 · 16 + 1 · 8 + 1 · 4 + 1 · 2 + 1 · 1 = 127
十进制 : 127
八进制 : 177 (127 = 2 · 64 + ... → 1 7 7)
十六进制 7楼 (127 = 7 · 16 + 15)

该工具包将数字视为无符号 32 位整数，因此有效范围为 0 到 4,294,967,295。十六进制输出以大写字母显示。

4.5 ASCII/HEX转换器

此工具显示每个字符的数字代码。在“文本→十六进制”模式下，每个字符都会被替换为其两位十六进制代码；在“十六进制→文本”模式下，则相反。由于它以一个字节为单位处理每个字符，因此仅接受 ASCII 字符（代码 00-7F）。

示例：你好→48 65 6C 6C 6F (H=0x48, e=0x65, l=0x6C, l=0x6C, o=0x6F)

5. 人脉拓展

这是大多数应用程序开发人员不太熟悉的部分，因此我们将对其进行最详细的讲解。这里的所有内容都与 IPv4 相关——即经典的四位数字地址。

5.1 IPv4 地址

IPv4 地址是一个 32 位数字，通常写成四个十进制“八位字节”（或称字节），用点号分隔，每个字节的取值范围为 0 到 255。例如 192.168.1.10。这四个八位字节就是一个 32 位值的四个字节。

将地址转换为单个 32 位整数是使所有计算变得简单的关键技巧——对整数进行算术运算，然后再转换回来。

192.168.1.10

= 192 · 256³ + 168 · 256² + 1 · 256 + 10 =

192 · 16777216 + 168 · 65536 + 1 · 256 + 10 =

3,232,235,786

二进制表示（32 位，每组一个字节）：

11000000 . 10101000 . 00000001 . 00001010

要转换回来，将 32 位值拆分为四个字节（分别向右移动 24、16、8 和 0 位，每次保留低 8 位）。

5.2 子网掩码和 CIDR 表示法

网络地址是一组连续的地址块。每个地址都分为网络部分（标识该地址块）和主机部分（标识该地址块内的设备）。CIDR 表示法用斜杠和数字来表示网络部分的大小，例如 /24，表示“前 24 位是网络部分”。

子网掩码表达的概念与 32 位值相同：网络部分全部为 1，主机部分全部为 0。

/24 → 11111111 11111111 11111111 00000000

= 255.255.255.0

/26 → 11111111 11111111 11111111 11000000

= 255.255.255.192

掩码的构造方式：取一个全为 1 的值（32 位），将其左移 (32 - 前缀) 位，使得左侧正好剩下“前缀”个 1 位。

5.3 CIDR 计算步骤

给定一个 IP 地址和一个前缀长度 /n，该计算器会生成六个结果。以下是完整的计算方法。

5. 将 IP 地址转换为 32 位形式（参见 5.1）。

6. 构建子网掩码：n 个 1 位，后跟 (32 - n) 个 0 位。

7. 网络地址 = IP AND 掩码。AND 运算会将每个主机位清零，只留下该块的起始地址。

8. 广播地址 = 网络地址 OR (NOT 掩码)。NOT 掩码对于网络部分为 0，对于主机部分为 1，因此 OR 运算会将每个主机位设置为 1——即该块的最后一个地址。

9. 第一个主机 = 网络地址 + 1（网络地址本身不分配给设备）。

10. 最后一个主机 = 广播地址 - 1（广播地址已保留）。

11. 可用主机数 = 2^(32 - n) - 2。块中的地址总数为 2^(32 - n)；减去 2 是网络地址和广播地址。

示例 1 — 192.168.1.10 /24

IP : 11000000 10101000 00000001 00001010 : (192.168.1.10)

口罩/24 11111111 11111111 11111111 00000000 (255.255.255.0)

网络 = IP 和掩码

: 11000000 10101000 00000001 00000000 广播 → 192.168.1.0

= 网络 OR (非掩码)

: 11000000 10101000 00000001 11111111 第一 → 192.168.1.255
个主机: 192.168.1.1 最后一个主机: 192.168.1.254
可用主机数 = $2^{(32-24)} - 2 = 2^8 - 2 = 254$

示例 2 — 172.16.5.130 /27（八位字节内的子网划分）

当前缀不是 8 的倍数时，分割发生在一个八位字节内。/27 掩码的最后一个八位字节以 11100000 结尾，即 224，因此最后一个八位字节的掩码为 255.255.255.224。“块大小”为 $256 - 224 = 32$ ，这意味着每个子网的宽度为 32 个地址：.0、.32、.64、.96、.128、.160、.192、.224。地址 .130 位于 .128-.159 块内。

IP :10000010 (最后一个八位字节 130)
口罩/27 :11100000 (255.255.255.224, 块大小为 32)

网络 = 172.16.5.128 (130 和 224 = 128)
广播地址 = 172.16.5.159 首台主 (128 + 32 - 1)
机: 172.16.5.129 最后一个主机: 172.16.5.158
可用主机数 = $2^{(32-27)} - 2 = 2^5 - 2 = 30$

最后一个字节的简便方法：块大小 = $256 - (\text{最后一个字节掩码值})$ 。网络的最后一个字节是小于等于地址最后一个字节的块大小的最大倍数。

子网掩码快速参考

CIDR	子网掩码	块大小	可用主机
/8	255.0.0.0	16,777,216	16,777,214
/16	255.255.0.0	65,536	65,534
/24	255.255.255.0	256	254
/25	255.255.255.128	128	126
/26	255.255.255.192	64	62
/27	255.255.255.224	32	30
/28	255.255.255.240	16	14
/29	255.255.255.248	8	6
/30	255.255.255.252	4	2

5.4 IP地址计算器

一旦地址是 32 位整数，算术运算就很简单，八位字节之间的进位会自动发生，因为它实际上是一个很大的数字。

- 加/减：转换为整数，加上或减去该值，然后转换回整数。加法运算结果不得超过整数值。255.255.255.255（溢出）且减去后不能低于 0.0.0.0（下溢）。
- 计数：从起始 IP 地址到结束 IP 地址（包括起始 IP 地址和结束 IP 地址）的地址数为（结束 IP 地址 - 起始 IP 地址）+ 1。

添加 : 192.168.1.250 + 10
250 + 10 = 260 → 进位到第三个八位字节的结果：
192.168.2.4
数数 : 192.168.1.1 至 192.168.1.10
(10 - 1) + 1 = 10 个 IP 地址

5.5 端口（端口检查器）

一台计算机可以同时运行多个网络服务。端口号（0-65535）用于标识连接所对应的服务。端口 0-1023 是“常用”端口，保留给常见的服务使用。

此工具包中的检查屏幕仅为线框图：浏览器或沙盒应用程序无法打开原始 TCP 连接，因此不会执行真正的检查。

服务	端口	服务	端口
HTTP	80	SSH	22
HTTPS	443	FTP	21
DNS	53	SMTP	25

6. 通用工具

6.1 UUID 生成器

UUID（通用唯一标识符）是一个128位数字，用作标识符，可以在多台机器上独立生成，几乎不会发生冲突。它由32个十六进制数字组成，分为五组，用连字符分隔：8-4-4-4-12。

xxxxxxxx-xxxx-4xxx-yxxx-xxxxxxxxxxxx

^

变体 nibble: 8、9、a、b 之一

版本 nibble: 4（基于随机的 UUID）

版本 4 的 UUID 由随机位填充，但有两个固定标记：版本号始终为“4”，第四组的第一位数字为 8、9、a 或 b。输出为小写，并且连续两代 UUID 实际上永远不会相等。

6.2 密码生成器

密码强度以熵来衡量，单位为比特。每增加一个字符，可能性数量就会乘以字符池的大小，因此熵会随着密码长度和字符种类的增加而增加：

熵（比特）≈ 长度 × log2（池大小）

泳池尺寸：小写字母 26，+大写字母52，+数字62，+符号≈94

生成器允许用户选择字符串长度（4-64 位）以及要包含的字符集。为了确保每个选择都得到尊重，结果保证至少包含每个选定字符集中的一个字符，并且只包含选定字符集中的字符。

6.3 正则表达式测试器

正则表达式（regex）是一种用于描述文本的简洁模式语言。测试器会报告测试字符串是否与某个模式匹配。此工具包执行部分搜索：当在测试字符串的任何位置找到该模式时，结果为“匹配”。要要求整个字符串都匹配，请在模式开头添加“^”，在结尾添加“\$”。

令牌	意义	令牌	意义
... \$	字符串的开始……结束	\d	数字 0-9
。	任何一个角色	\w	一个单词字符
*	零或以上	+	一个或多个
{m,n}	介于 m 和 n 次之间	[abc]	任何列出的角色

例子 `a-zA-Z0-9_]{6,16}$` 匹配由 6 到 16 个字母、数字或下划线组成的用户名。

6.4 随机数生成器

该生成器生成均匀分布在指定范围内的整数。范围 $[\min, \max]$ 内的值是通过将原始随机数进行映射得到的： $\min + \text{random} \cdot (\max - \min + 1)$ 。每个值都是独立的，因此可能出现重复值，并且允许正负值。对于安全要求较高的应用（例如密钥、盐值、密码），应使用加密安全的随机源，而不是普通的随机源。

附录A. 快速参考表

A.1 二的幂

n	2 ⁿ	n	2 ⁿ
0	1	10	1,024
1	2	16	65,536
4	16	24	16,777,216
8	256	32	4,294,967,296

A.2 十六进制↔二进制半字节

十六进制	二进制	12月	十六进制	二进制	12月
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	一个	1010	10
3	0011	3	B	1011	11
4	0100	4	C	1100	12
5	0101	5	D	1101	13
6	0110	6	E	1110	14
7	0111	7	F	1111	15

A.3 部分可打印 ASCII 码

查尔	十六进制	12月	查尔	十六进制	12月
(空间)	20	32	一个	41	65
0	30	48	H	48	72
9	39	57	一个	61	97
@	40	64	o	6F	111

A.4 样本测试向量中使用的值

这些输入和输出出现在应用程序原型和 test-vectors.json 文件中。它们可用于检查实现情况。

工具	输入	预期输出
MD5	世界技能大赛	e514b0a3298d6efe1322b85ddb91a791
SHA-256	世界技能大赛	7a474640...387ffd75 (64 十六进制)
HMAC-SHA256	世界技能大赛, 关键秘密	dd9e9b2e...22ce6ee8 (十六进制 64)
AES-128 CBC	你好, 世界技能大赛	n2tQ3xoA0rRAUolnZAeHLBjD4S6g0VhQwrHfkXcl/Hw=
Base64	2026年世界技能大赛	V29ybGRTa2lsbHMgMjAyNg==
CIDR /24	192.168.1.10/24	net .0, 广播 .255, 254 主机
IP地址	192.168.1.250 + 10	192.168.2.4

技术参考指南结束 —